

FLite - FORTRAN Inspired Compiler with Algorithmic NLP Diagnostics

Abstract

This report presents the design and implementation of F-Lite, a compiler for a custom FORTRAN-inspired Domain Specific Language (DSL). Developed in C++14, the compiler follows a traditional multi-phase pipeline: Lexical Analysis (DFA-based scanner), Recursive Descent Parsing, Intermediate Code Generation via Three-Address Code (TAC), and Target Code Generation to a simplified assembly language. A key novelty is the integration of "Light NLP" diagnostics -- specifically Levenshtein Distance for keyword recovery and Bigram-based Syntax Hinting -- to provide intelligent, human-readable error messages within a line-numbered listing file. The project also includes an interactive web-based visualizer (HTML/JS/jQuery) that lets users step through each compiler phase and observe how source code is transformed at every stage. The system is intentionally "barebones," avoiding generative tools like Flex/Bison to demonstrate deep subject knowledge in compiler theory.

1. Introduction

1.1 Motivation

FORTRAN (FORmula TRANslation) is historically significant as one of the first high-level programming languages. The F-Lite language captures the essential spirit of FORTRAN -- imperative, line-based execution with formula translation -- while simplifying the grammar for pedagogical clarity.

Most student compiler implementations rely on automated generator tools (Flex/Bison, ANTLR) and provide only rudimentary error messages. F-Lite diverges on both fronts:

1. **Manual Implementation:** Every phase of the compiler is hand-coded, including a DFA-based lexical analyzer and a recursive descent parser.
2. **NLP-Enhanced Diagnostics:** The compiler uses computational linguistics algorithms (Levenshtein Distance, Bigram Analysis) to produce intelligent error recovery suggestions.

1.2 Language Overview

F-Lite is an imperative, line-based, case-insensitive language supporting:

- **Data Types:** 32-bit integers and 1D integer arrays
- **Control Flow:** IF/THEN/ELSE conditionals and DO loops
- **I/O:** READ and WRITE statements
- **Modularity:** Subroutines with parameter passing
- **Comments:** Line comments beginning with `!`
- **Strings:** Apostrophe-delimited with escape sequences (`\n`, `\t`, `\'`)

1.3 Reserved Keywords

PROGRAM , INTEGER , READ , WRITE , IF , THEN , ELSE , DO , END , SUBROUTINE , CALL , RETURN ,
STOP

2. Implementation Description

2.1 Architecture Overview

The compiler is structured as a pipeline with four primary phases:

```
Source (.fl)
  |
  v
[Lexical Analyzer] ----> Token Stream
  |
  v
[Parser + NLP Engine] ----> TAC + Diagnostics
  |
  v
[Code Generator] ----> Target Assembly (.asm)
  |
  v
[Listing Manager] ----> Listing File (.lst)
```

2.2 Lexical Analyzer (scanner.cpp)

The scanner implements a manual DFA (Deterministic Finite Automaton) using a function-dispatch architecture. Rather than an explicit state transition table, each token category is handled by a dedicated scanning function:

- `scan_identifier()` : Accumulates alphanumeric characters and underscores; performs case-insensitive keyword lookup via hash map.
- `scan_number()` : Accumulates digit sequences into integer literals.
- `scan_string()` : Handles apostrophe-delimited strings with escape sequence processing.
- `scan_dot_operator()` : Recognizes FORTRAN-style relational operators (`.EQ.` , `.LT.` , etc.) using 2-character lookahead.

Case Insensitivity: Identifiers are converted to uppercase for keyword matching and symbol table operations, preserving the original case in the token's lexeme for diagnostic output.

Identifier Uniqueness: Following the specification, identifiers are considered unique based on their first 32 characters (after case normalization).

Token Types: The system defines approximately 35 distinct token types covering keywords, arithmetic operators, relational operators (both symbolic and dot-notation), delimiters, literals, and special tokens (NEWLINE, EOF, UNKNOWN).

2.3 Recursive Descent Parser (parser.cpp)

The parser implements a top-down recursive descent strategy with one function per grammar non-terminal. The grammar is defined in EBNF in `docs/grammar.ebnf` .

Expression Parsing: Operator precedence is encoded structurally:

- `parse_expression()` handles addition/subtraction (lowest precedence)
- `parse_term()` handles multiplication/division
- `parse_factor()` handles atoms, unary operators, and parenthesized expressions (highest precedence)

Error Recovery: The `synchronize()` method implements panic-mode recovery. On encountering a syntax error, the parser skips tokens until it finds:

1. A NEWLINE token (line boundary), or
2. A keyword that begins a new statement (`IF` , `DO` , `READ` , `WRITE` , `END` , etc.)

This allows the compiler to report multiple errors per compilation run rather than stopping at the first error.

Syntax-Directed Translation: The parser emits Three-Address Code (TAC) instructions inline during parsing, eliminating the need for a separate AST construction phase.

2.4 Light-NLP Diagnostic Engine (`nlp_engine.cpp`)

This is the key novelty of the F-Lite compiler. The diagnostic engine integrates two computational linguistics algorithms:

2.4.1 Levenshtein Distance

The Levenshtein Distance (edit distance) measures the minimum number of single-character edits (insertions, deletions, substitutions) required to transform one string into another.

Algorithm: The implementation uses the classic Wagner-Fischer dynamic programming approach with a two-row space optimization, reducing memory from $O(m*n)$ to $O(n)$:

```
Given strings a[1..m] and b[1..n]:
  Initialize prev[0..n] = 0, 1, 2, ..., n
  For i = 1 to m:
    curr[0] = i
    For j = 1 to n:
      cost = 0 if a[i] == b[j], else 1
      curr[j] = min(prev[j] + 1,          // deletion
                    curr[j-1] + 1,       // insertion
                    prev[j-1] + cost)    // substitution
    Swap prev, curr
  Return prev[n]
```

Application: When the parser encounters an unknown token at statement level, or when an identifier appears where a keyword is expected, the engine computes the Levenshtein distance against all reserved keywords. If the minimum distance is ≤ 2 , a "Did you mean 'X'?" suggestion is generated.

Example: The misspelling `INTGER` has a Levenshtein distance of 1 from `INTEGER` (one transposition modeled as delete + insert), producing: unknown keyword '`INTGER`' -- Did you mean '`INTEGER`'?

2.4.2 Bigram-Based Syntax Hinting

A bigram model captures which token types commonly follow which other token types, derived from the F-Lite grammar. This is represented as a hand-coded lookup table mapping each `TokenType` to a human-readable description of expected successors.

Example Entries:

Previous Token	Expected After
<code>KW_IF</code>	<code>'('</code> followed by a condition
<code>KW_INTEGER</code>	a variable name or array declaration
<code>OP_ASSIGN</code>	an expression
<code>IDENTIFIER</code>	<code>'=</code> ', <code>'('</code> , <code>'</code> , <code>'</code> , an operator, or newline

Application: When a syntax error occurs, the engine looks up the previous token's type in the bigram table and appends a contextual hint: Hint: after `IDENTIFIER`, typically `'=`', `'('`, `'`, `'`, an operator, or newline follows.

2.5 Symbol Table (`symbol_table.hpp`)

The symbol table is implemented as a header-only component using a vector-based store with linear scope search. It supports:

- **Scope Management:** `enter_scope()` / `exit_scope()` for subroutine-level scoping
- **Redeclaration Detection:** Prevents duplicate declarations within the same scope
- **Symbol Kinds:** `VARIABLE`, `ARRAY`, `SUBROUTINE`, `PROGRAM`

2.6 Intermediate Code Generation

The compiler produces Three-Address Code (TAC) as its intermediate representation. Each instruction has at most three operands:

- **Arithmetic:** `t0 = X + Y`
- **Assignment:** `SUM = t0`
- **Array Access:** `t1 = A[I]` and `A[I] = t1`
- **Control Flow:** `if X > N goto L1, goto L0`
- **I/O:** `read X, write X, write "string"`
- **Procedures:** `param X, call SUB, 2, return`

2.7 Target Code Generation (`codegen.cpp`)

The target code generator translates TAC into a simplified assembly language with the following instruction set:

Instruction	Description
LOAD Rn, var	Load variable into register
STORE var, Rn	Store register to memory
ADD/SUB/MUL/DIV Rd, Rs1, Rs2	Arithmetic on registers
LOADARR Rd, arr, Ri	Load array element
STOREARR arr, Ri, Rs	Store to array element
CMP R1, R2	Compare and set flags
JEQ/JNE/JLT/JGT/JLE/JGE label	Conditional jump
JMP label	Unconditional jump
READ Rn	Read input to register
WRITE Rn	Write register to output
WRITES "str"	Write string literal
PUSH/CALL/RET	Procedure support
HALT	Program termination

The generator uses a simple register allocation scheme with four general-purpose registers (R1-R4).

2.8 Listing Manager (`logger.cpp`)

The listing manager generates a `.lst` file that interleaves source lines with diagnostic messages:

```

1 | PROGRAM ERRORS
2 |   INTGER X
  | *** unknown keyword 'INTGER' -- Did you mean 'INTEGER'?
3 |   INTEGER Y

```

Errors are sorted by line number and placed immediately after their corresponding source line. Multi-line diagnostic messages (with hints) are indented for readability.

2.9 Interactive Web Visualizer (`web/`)

To make the compiler accessible as a teaching tool, the entire compiler pipeline was ported to JavaScript and embedded in a single-page web application. The visualizer allows users to step through each compiler phase interactively without installing any tools.

Architecture: The web app consists of four files:

- `index.html` - Page structure with a split-panel layout (source code on the left, phase visualization on the right)
- `css/style.css` - Dark theme styling with color-coded token categories
- `js/compiler.js` - A faithful JavaScript port of all C++ compiler components (Scanner, NLP Engine, Parser, SymbolTable, TargetCodeGen)

- `js/app.js` - UI logic handling phase tabs, rendering, and navigation

Phase Walkthrough: The app presents five phases accessible via tabs and Prev/Next navigation:

1. **Source** - A code editor pre-loaded with an example program. Users can write or modify F-Lite code directly in the browser.
2. **Tokens** - The source code is displayed with inline syntax highlighting (keywords in red, identifiers in blue, numbers in orange, strings in light blue, operators in red). The right panel shows a complete token table with type badges, lexemes, and positions.
3. **Parser + NLP** - On success, a green confirmation is shown. On failure, each error is displayed as a card with the line number, error message, Levenshtein suggestion (with edit distance badge), and bigram hint. Source lines with errors are highlighted in red.
4. **TAC** - The Three-Address Code output is displayed with color-coding by instruction type: labels in purple, arithmetic in red, assignments in blue, jumps in orange, I/O in green.
5. **Assembly** - The target assembly is displayed with instruction categories colored: loads in blue, stores in green, arithmetic in orange, jumps in red, I/O in green.

Design Decisions:

- The compiler runs entirely in the browser with no backend server required. A local HTTP server is only needed to avoid browser CORS restrictions when loading from `file://`.
- jQuery is used for DOM manipulation, loaded from CDN.
- The JavaScript compiler is a direct port of the C++ implementation, preserving the same class structure, method names, and algorithm logic. This ensures identical behavior between the CLI and web versions.
- The `compile()` function returns a structured result object containing tokens, errors, TAC instructions, assembly lines, and success status, making each phase independently renderable.

3. Results

3.1 Successful Compilation

For a valid program (`tests/valid_program.fl`) that reads an array, computes its sum, and prints the result:

- **Listing:** Clean listing with 0 errors
- **TAC:** 28 three-address instructions including loop control, array access, and I/O
- **Assembly:** 57 target instructions with proper register allocation and label management

3.2 Error Detection and NLP Diagnostics

For an error-laden program (`tests/error_program.fl`) containing:

1. Misspelled keyword (`INTGER` instead of `INTEGER`)
2. Incomplete expression (`X = 10 +`)
3. Missing closing parenthesis in IF condition
4. Unterminated string literal

The compiler detected 6 errors with NLP-enhanced messages including:

- **Levenshtein Recovery:** unknown keyword 'INTGER' -- Did you mean 'INTEGER'?
- **Bigram Hinting:** Hint: after IDENTIFIER, typically '=', '(', ',', an operator, or newline follows.
- **Continuation:** All 6 errors reported in a single compilation pass

3.3 Web Visualizer

The interactive web app successfully demonstrates all compiler phases:

- **Token visualization:** All tokens are correctly color-coded by category in the source view, with a complete token table showing type, lexeme, line, and column
- **NLP error display:** Levenshtein edit distance badges appear alongside "Did you mean?" suggestions, and bigram hints are rendered below each error
- **TAC and Assembly:** Both intermediate and target code outputs are rendered with category-based color-coding, matching the CLI compiler output exactly
- **Cross-platform:** The visualizer runs in any modern browser (Chrome, Firefox, Edge) with no installation required

4. Discussion of NLP Novelties

4.1 Levenshtein Distance: Mathematical Foundation

The Levenshtein Distance $d(a, b)$ satisfies the properties of a metric:

1. **Non-negativity:** $d(a, b) \geq 0$
2. **Identity:** $d(a, b) = 0$ iff $a = b$
3. **Symmetry:** $d(a, b) = d(b, a)$
4. **Triangle Inequality:** $d(a, c) \leq d(a, b) + d(b, c)$

The time complexity is $O(mn)$ where m and n are the string lengths. For keyword comparison (max keyword length 10), this is effectively $O(1)$ per comparison. The two-row optimization reduces space from $O(mn)$ to $O(n)$.

Threshold Selection: A distance threshold of 2 was chosen empirically. At distance 1, common typos (transpositions, single omissions) are caught. At distance 2, slightly more garbled inputs are recovered. Beyond 2, false positives increase significantly given the small keyword set.

4.2 Bigram Model: Grammar-Derived Expectations

Unlike statistical NLP bigram models trained on corpora, our bigram table is deterministically derived from the formal grammar. This guarantees that suggestions are always syntactically valid. The model operates on token types rather than lexemes, providing abstract syntactic guidance rather than specific token predictions.

Advantages over traditional error messages:

- **Context-aware:** Messages reference what was expected based on what preceded the error

- Educational: Hints teach the language syntax alongside reporting errors
- Concise: Human-readable descriptions rather than grammar rule dumps

4.3 Integration Strategy

The NLP engine operates at two levels:

1. **Lexical Level:** During statement parsing, identifiers that resemble keywords (Levenshtein distance ≤ 2) and appear in keyword positions (not followed by `=` or `(`) trigger keyword recovery suggestions.
2. **Syntactic Level:** When the parser's `expect()` fails, the `diagnose()` function combines the expected token, the found token, and the previous token to produce a composite error message with both Levenshtein suggestions and bigram hints.

5. Conclusion

The F-Lite compiler demonstrates that a complete compiler chain -- from lexical analysis through target code generation -- can be implemented manually in C++ without reliance on generator tools. The integration of Levenshtein Distance and Bigram-based diagnostics produces error messages that are significantly more helpful than traditional "expected X, found Y" reporting.

Key achievements:

- **Complete Pipeline:** Scanner, parser, ICG (TAC), target code generation, and listing management
- **Robust Error Recovery:** Panic-mode recovery enables multi-error reporting per compilation
- **NLP Diagnostics:** Computational linguistics algorithms provide "Did you mean?" suggestions and contextual syntax hints
- **Interactive Visualizer:** A browser-based teaching tool that walks through each compiler phase with color-coded output and step-by-step navigation
- **Dual Implementation:** The compiler exists as both a C++14 CLI tool and a JavaScript web app, with identical behavior across both
- **Clean Architecture:** Modular design with clear separation of concerns across 6 source files, 5 header files, and 4 web files
- **No External Dependencies:** Pure C++14 CLI with no third-party libraries or generator tools; web app uses only jQuery

Future Work

- Extend the type system to support REAL (floating-point) and CHARACTER types
- Implement a proper register allocator using graph coloring
- Add optimization passes on the TAC (constant folding, dead code elimination)
- Extend NLP diagnostics with longer n-gram context for improved suggestions
- Add animated token-by-token stepping in the web visualizer
- Add parse tree visualization as a collapsible tree diagram